

Session 5 · Deck 5.B1

Projection: Making a Residual Small

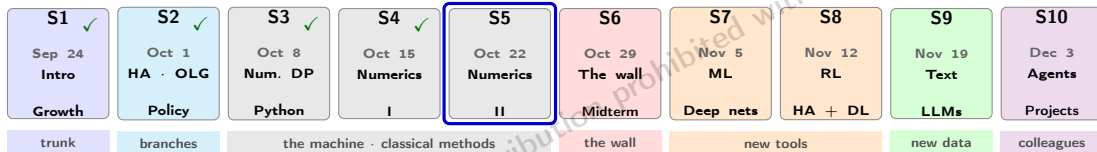
Zhigang Feng

Thursday, October 22, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 5.B1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 5.B1 the residual · 5.B2 choosing a basis · 5.B3 Chebyshev · 5.B4 finite elements · 5.B5 determining the coefficients · 5.B6 the worked example.

The Other Road

Block A gave up **global validity** to buy speed and dimension. We measured the cost: nothing at the steady state, 1.8 decades at half of it, and everything at a kink.

Block B makes the opposite trade. Keep the solution valid on the **whole domain**, constraints included — but stop storing one number per grid point.

The unknown becomes a short vector of **coefficients**, and “solved” is redefined: not “the value function stopped moving”, and not “the derivatives match at a point”, but **the residual is zero**.

Method	The unknown is
VFI (S3–S4) perturbation projection	N values on a grid derivatives at one point coefficients on a basis

And there is a fourth row waiting in S7: a neural network is this same move with a different, learned basis.

The Model, With Labour This Time

$$\text{(Euler)} \quad \frac{1}{c_t} = \beta \mathbb{E}_t \left[\frac{1}{c_{t+1}} \left(1 + \alpha k_{t+1}^{\alpha-1} (e^{z_{t+1}} l_{t+1})^{1-\alpha} - \delta \right) \right]$$

$$\text{(Labour)} \quad \psi \frac{c_t}{1 - l_t} = (1 - \alpha) k_t^\alpha (e^{z_t} l_t)^{1-\alpha} \frac{1}{l_t}$$

$$\text{(Resource)} \quad c_t + k_{t+1} = k_t^\alpha (e^{z_t} l_t)^{1-\alpha} + (1 - \delta) k_t$$

$$\text{(Shock)} \quad z_t = \rho z_{t-1} + \varepsilon_t$$

The objective is unchanged from 5.A1: find d with $H(d) = 0$, where now

$$k_{t+1} = d_1(k_t, z_t), \quad l_t = d_2(k_t, z_t).$$

Read the indices

Everything inside the $t + 1$ expectation carries $t + 1$ subscripts — including the shock and labour inside the marginal product. Getting that wrong is the commonest transcription error in this literature, and it survives compilation.

Endogenous labour is here to make the point that projection handles *systems*: two unknown functions, two residuals, one co-efficient vector.

The Core Idea, in One Line

Restrict the unknown function to a finite-dimensional family:

$$d^n(x; \theta) = \sum_{i=0}^n \theta_i \psi_i(x)$$

- $\{\psi_i\}$ is a chosen **basis** — polynomials, splines, orthogonal polynomials, tent functions.
- $\{\theta_i\}$ are the **unknown coefficients**, and there are finitely many.

Substituting into the equilibrium conditions defines the **residual function**

$$R(x; \theta) = H(d^n(x; \theta)),$$

which is what we try to make small.

Quant_Macro Lec. 9, "Projection: The Core Concept".

Two separate choices

- (1) which basis $\{\psi_i\}$ — 5.B2, 5.B3, 5.B4.
- (2) what "small" means — 5.B5. Different answers to these two questions are the different names in the literature. The framework is one framework.

Replace $\sum \theta_i \psi_i$ by $\mathcal{N}(x; \theta)$, a neural network, and the framework is unchanged. That is S7.

Why the Name “Projection”

The true d lives in a big function space — C^1 , or L^2 .

Choosing a basis **projects** that infinite-dimensional problem onto a finite-dimensional subspace.

We then cannot expect $H(d^n) = 0$ *everywhere*; there are only $n + 1$ coefficients. Instead we impose finitely many conditions:

- **Collocation:** $R(x_j; \theta) = 0$ at $n + 1$ chosen points.
- **Galerkin:** $\langle R(\cdot; \theta), \psi_i \rangle = 0$ for each i — the residual is orthogonal to the basis.
- **Least squares:** minimise $\|R(\cdot; \theta)\|$ over the domain.

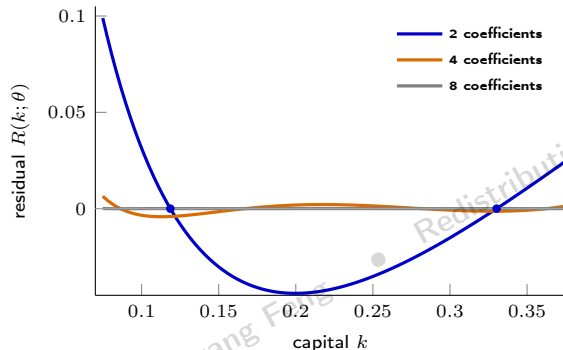
Each is a sense in which the residual has been *projected away*.

Quant_Macro Lec. 9, “Why ‘Projection’? Mathematical Intuition”.

The result is **global**: valid over the whole domain, not a neighbourhood — provided the basis can represent the function and the projection is sensibly chosen.

Compare 5.A3: perturbation imposes conditions on *derivatives at one point*. Projection imposes conditions on *values over a domain*. Same residual, opposite discipline.

The Residual Is a Function — Look at It



Brock–Mirman, deterministic, Chebyshev in k , coefficients chosen by collocation.

The residual is exactly zero at the collocation nodes (dots, 2×10^{-16}) and free everywhere else. More coefficients push the whole curve towards the axis.

coeff.	$\max R $	policy error
2	0.0990	5.96%
4	0.0064	0.36%
8	5.7×10^{-5}	0.0038%

Ledger projection residual. Policy error is $\max |g/g^* - 1|$ against the closed form $\alpha \beta k^\alpha$ on $[0.4k^*, 2k^*]$.

And Notice What the Residual Is Doing

In every row of that table the residual is **larger** than the policy error it is standing in for: 0.099 against 0.060, 0.0064 against 0.0036, 5.7×10^{-5} against 3.8×10^{-5} .

That is not a coincidence. It is the property 3.A1 needed from Santos (2000): the residual is **computable** and the policy error is **not**, and the theorem says the second is bounded by a constant times the first.

Here, with the closed form in hand, we can check the bound directly instead of trusting it — and the ratio sits between 1.5 and 1.8 at every basis size.

Ledger projection_residual; Santos (2000), and Santos & Vigo-Aguiar (1998) for the h^2/h pair.

So “make the residual small” is not a convenient proxy chosen for tractability. It is the only error measure you can compute, and there is a theorem licensing it.

The constant depends on the curvature of u and grows as the problem flattens. A residual is an *order of magnitude*, not an error bar (3.A1).

The Algorithm, Formally

1. Define $n + 1$ linearly independent basis functions $\psi_i : \Psi \rightarrow \mathbb{R}^m$.
2. Define the coefficient vector $\theta = [\theta_0, \dots, \theta_n]$.
3. Form the approximant $d^n(\cdot; \theta) = \sum_{i=0}^n \theta_i \psi_i(\cdot)$.
4. Substitute into H to get the residual

$$R(\cdot; \theta) = H(d^n(\cdot; \theta)).$$

5. Choose $\hat{\theta}$ to make it small under a metric ρ :

$$\hat{\theta} = \arg \min_{\theta} \rho[R(\cdot; \theta)].$$

Quant_Macro Lec. 9, "Algorithm".

Where the work is

Step 5 is a **nonlinear system** in θ , solved by Newton or a quasi-Newton method — so all of 4.A1 and 4.A3 apply here directly.

Projection Against Perturbation

	Perturbation (Block A)	Projection (Block B)
Approximates	locally, around one point	globally, on a domain
Unknowns are	derivatives at that point	coefficients on a basis
Conditions	derivatives of F vanish	R small at points, or in an inner product, or in norm
Speed	very fast; analytic derivatives	a nonlinear solve; slower
Dimension	tractable in large state spaces — only local information is used	the curse of dimensionality returns
Nonlinearity	degrades away from the point	handled directly; no expansion

Not rivals

They mix. Perturbation supplies the **initial guess** for the projection solve — which is exactly what 5.B6 does, and why its nonlinear system converges at all.

Takeaways

- Projection replaces the unknown *function* with a finite vector of **coefficients on a basis**, and redefines “solved” as **the residual is small**.
- Two independent choices: **which basis** (5.B2–5.B4), and **what “small” means** (5.B5). Every named method in the literature is one pair of answers.
- The residual is a **function**. Collocation zeroes it at $n + 1$ points and leaves it free in between — measured: 2×10^{-16} at the nodes, 0.0990 between them with two coefficients.
- More coefficients push the whole curve down: $0.0990 \rightarrow 0.0064 \rightarrow 5.7 \times 10^{-5}$ for 2, 4, 8 coefficients, with policy error falling in step.
- The residual **exceeded** the policy error in every case, by a factor of 1.5 to 1.8 — the property that makes Santos’s theorem usable, checked here against a closed form rather than assumed.
- The solution is **global**: valid on the whole domain, kinks and constraints included, which is precisely where Block A had nothing to say.

Bridge: Which Basis?

Everything above was written with an unspecified $\{\psi_i\}$, and the example quietly used Chebyshev polynomials without saying why.

The choice matters more than it looks. The obvious basis — $1, x, x^2, \dots$ — fails, and it fails for a reason that has nothing to do with approximation theory and everything to do with conditioning.

Next (5.B2)

Spectral against finite-element bases; why monomials are unusable; orthogonality; and the Gibbs phenomenon, which decides what happens when the function you are approximating has a kink.